Problem Statement :→ Distributed Web Crawler

Note : Avoid directly jumping to solution

If not sure about problem statement – talk to interviewer – can you please elaborate – what it should do?

Steps

① Requirement Gathering / scope of the problem. (that can be solved in 40-45 min)

↓ Functional Requirements (What client needs)

Non-Functional Requirements (Performance, scalability, monitoring etc)

② Data / Numbers < storage, computation } Web crawler

→ 1 bn websites (no pages / websites)

→ 100 bn webpages (100 Kb data)

→ 100 bn × 100 Kb

= 100 GB × 100 × 1000

= 10^7 GB.

= 10 PB

③ Components

↳ Loosely coupled components

(Architectural diagram & scope of components)

④ Communication b/w components < sync, async.

Web crawler → go to website, collect data and use it somewhere.

↳ Use case: Google. (indexed fashion)

① Requirement Gathering

→ What is web crawler? → Software that crawels from one website to another.

↓

starting from facebook.com

↓

scan all the website of this world

scan content + download.

google.com

linkedin.com

wikipedia.com

→ Why? What we want to solve?

① Use case: Search Engine

↓

Google keeps on crawling all web-sites & keeps them using indexing.

② Duplicacy check | Copyright Infringement.

③ Keyword based search.

④ Application: All the offers present on all diff^t e-commerce websites

All Steps

① Requirement : Create a web-crawler which can scan all the websites of this world.

Build ↓ Scope : (Pick few imp^t features)

↳ Two-way communication with interviewer → If already aware about topic – can design (suggest) scope & ask interviewer if its fine. Can ask interviewer to build scope.

Functional Requirement

(Crawl all web pages)

(X Downloading Duplicate content)

(Avoid visiting same URL) (Avoid infinite loop)

Non-Functional Requir / Considerations

(Websites are continuously ↑)

↓

- Scalable system
- Extensible system
- Easy to understand.
- Be polite to website [Make sure we are not using many concurrent crawlers – can bring down website]

(2) Estimation of infrastructure

(Every quarter - There is planning)

- Storage → 8 PB
- Compute
- N/W

Can't be done by single system → Distributed Solution

- Assumption : Websites 1 bn → 60% genuine

↳ Effective Websites : 0.6 bn.

Websites < wikipedia – 1000s pages
1-2

∴ Total pages = 0.6 × 100 bn
= 60 bn pages.

- Avg size of webpage = 100 Kb

Total size of data = 60 bn × 100 = 6bn × 10^3 Kb

= 6 bn × Bytes × 10^6

= $\boxed{6 \text{ PB}}$ → 75%

We need around 8 PB storage.

③ Steps

① Start with list of URLs — URL Provider

seed URLs

② Fetch HTML content of URL — Content Fetcher

③ Check for duplicate — Duplicate Checker

→ Duplicate – Ignore

No ↓

④ Download the content — Content Downloader

⑤ Parse the content & fetch new URLs — URL Parser

Continue

Each step can be broken down in single step.

URL Provider

Content Fetcher

Duplicate checker

URL Parser

Content Downloader

① URL Provider

Web Pages :→ 60 Bn URLs

Avg size of 1 URL → 100 Bytes

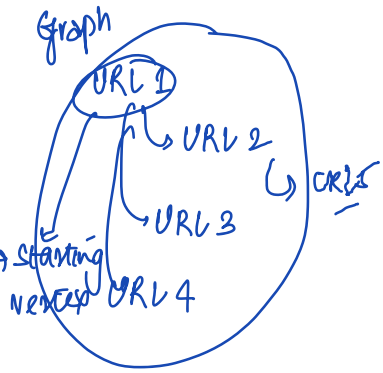
↓

Total - size = 60B × 100 Bytes

= 6GB × 10^3

= 6TB

Distributed



- Data structure to store URL : Complex Graph

↓

Traversing it → we follow BFS (first nearest then farther)

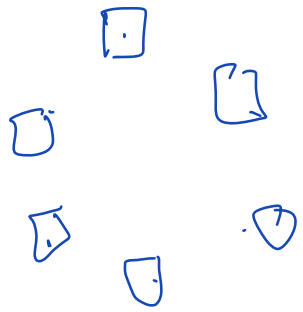
↓

Internally Queue data structure

(first in first out)

② Content Fetcher – Distributed

Challenge of multiple fetchers →



↓

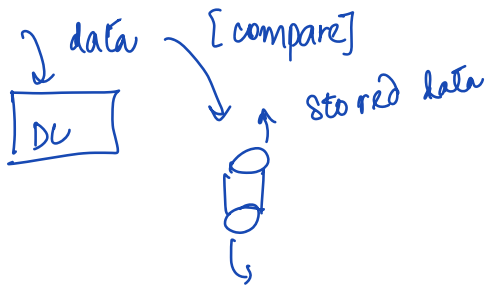
eg further URLs of fb →

Concurrency calls → can slow it down

+ Politeness effected

[Make sure all links belonging to same domain, should end up at same machine]

③ Duplicate Checker – distributed



How to compare?

↳ character per character compare (very slow)

Use hashing – hash new data & compare with hash of original data – comparatively very fast. → Challenge : Huge difference.

hash (abc145 CD MNO) → 34 – ..

hash (Abc145 CD MNO) → 51 – ..

Smart ways : AI/ML → 75% similarity.

④ Content Downloader → can temporarily put some data in External Cache (REDIS)

6PB ↑

horizontally scalable ?

↓

HDFS | S3 | Data Lakes

↓

So that when others want data - they don't have to deal with slow I/O hard disk operations

⑤ URL Parser

Scan downloaded document to get URLs.

↳ Check for new URLs

URL new.

List of old URLs → constant time ?

↓

Bloom Filters

↓

Data structure which tells if entry is present or not in constant time.

① Linear Search (O(N))

② Binary Search O(log n)

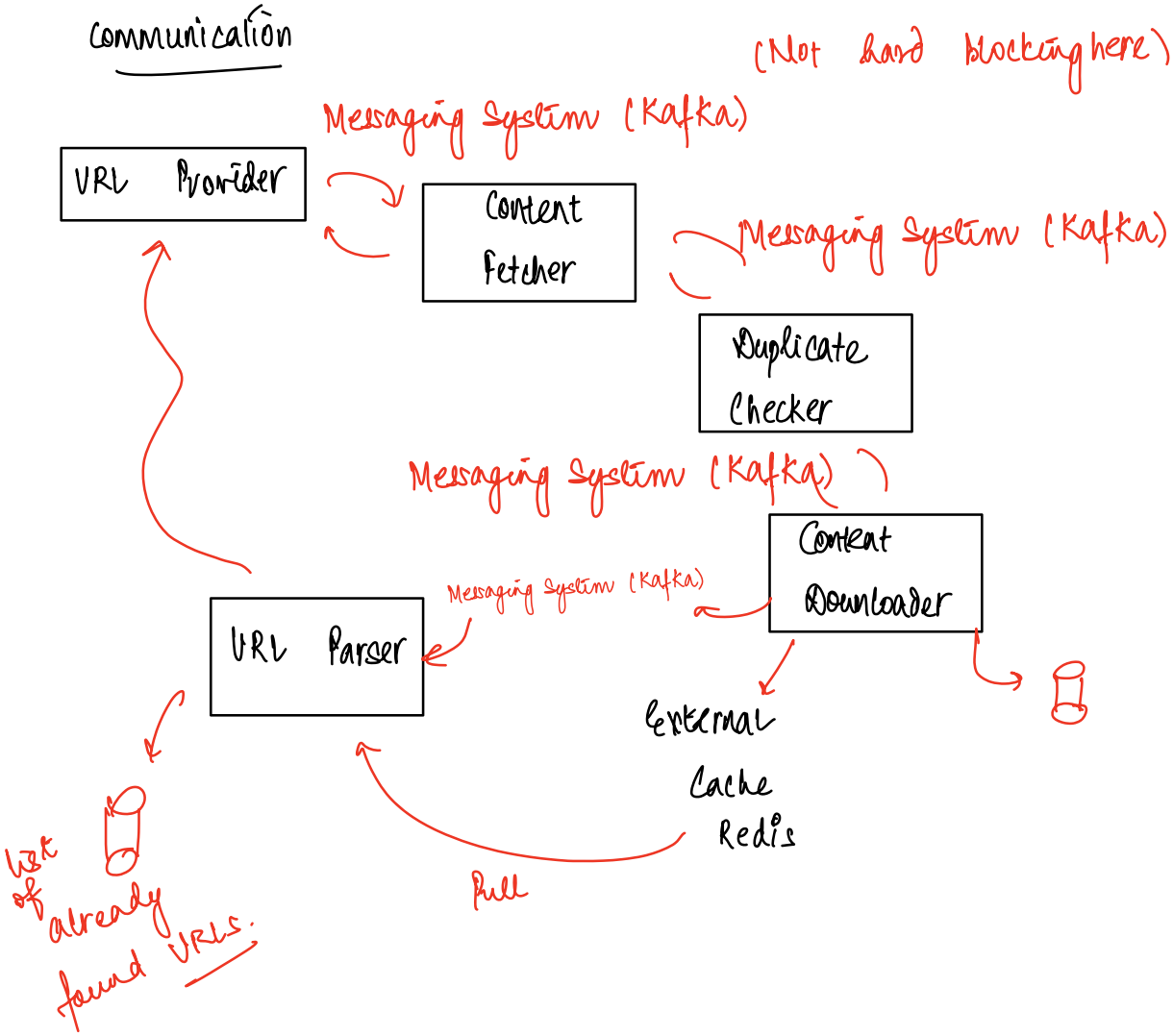
Has some constraints

↓

Can work will work little up down data.

Bloom Filter

(a) Key is not present → 100% Accuracy

(b) Key - Present → 80 - 100% { can miss some URLs }

Communication